



DÉROULÉ DE SESSION · SUPPORT FORMATEUR

Formation Prometheus & Grafana

Session Apicil (Lyon) — jeudi, la journée pratique

Sommaire

Session Apicil (Lyon) — jeudi, la journée pratique	2
La journée en un coup d'œil	2
Ce soir (mercredi) — 20 minutes	2
8h30 — Vérification	3
9h00 — Accueil et rattrapage	3
9h20 — Mission 0 : la chasse aux métriques	5
9h35 — Missions 1 à 8 : PromQL dans Grafana Explore	5
10h30 — Pause	8
11h00 — Grafana en 15 minutes	8
11h15 — TP A : le dashboard d'un serveur Linux	8
12h20 — Le dashboard de la communauté, puis débrief	9
12h30 — Déjeuner	10
13h30 — TP B : le dashboard de la boutique	10
14h30 — Pause	11
15h00 — TP C : alerter	11
15h35 — War game en binôme	12
15h50 — Clôture	13

Session Apicil (Lyon) — jeudi, la journée pratique

Ce document remplace, pour le jeudi, le déroulé prévu dans « Session Apicil (Lyon) — déroulé sur 21 h ». La situation : mercredi soir, on vient à peine d'attaquer l'instrumentation (module 6, TP 2). Le groupe est un groupe d'infrastructure, pas de développeurs : leur faire écrire les cinq TODO de `app.py` coûterait la matinée pour un bénéfice faible. Je leur **donne** l'application instrumentée et je consacre la journée à ce qu'ils utiliseront lundi : interroger, visualiser, casser, alerter.

Le principe de la journée tient en une phrase, que je dis à 9h00 : **aujourd'hui, vous ne codez pas, vous pilotez.**

Horaires : 9h00–16h00, déjeuner d'une heure, deux pauses d'une demi-heure. Il reste **cinq heures de travail effectif**, dont environ 3h45 les mains sur le clavier.

Supports :

- **Stagiaires** : *Guide stagiaire — Dernier jour, version pratique* (`docs/stagiaire/Guide-stagiaire-Jour-3-express.pdf` , et `jour-3-express.md` pour Notion). Il remplace le guide du jour 3 pour aujourd'hui ; le guide du jour 3 reste la référence pour le faire chez soi.
- **Moi** : ce document, le deck habituel (je donne les numéros de slides à passer, le reste je le saute), et mon Codespace de démonstration.

La journée en un coup d'œil

Heure	Durée	Séquence	Slides	Modalité
9h00	20 min	Accueil, rattrapage en une commande, ce qu'on aurait codé	41, 42	Tous ensemble, script
9h20	15 min	Mission 0 — La chasse aux métriques	22	Individuel
9h35	55 min	Missions 1 à 8 — PromQL dans Grafana Explore	48, 50, 53, 54	Individuel, correction au fil de l'eau
10h30	30 min	<i>Pause</i>		
11h00	15 min	Grafana en 15 minutes, démo d'un panel de bout en bout	59 à 65	Démonstration
11h15	65 min	TP A — Dashboard serveur Linux	66, 67	Individuel
12h20	10 min	Le dashboard de la communauté (ID 1860), débrief TP A		Tous ensemble
12h30	60 min	<i>Déjeuner</i>		
13h30	60 min	TP B — Dashboard de la boutique	68, 69	Individuel
14h30	30 min	<i>Pause</i>		
15h00	35 min	TP C — Une alerte Prometheus, puis une alerte Grafana	78, 81, 94	Démonstration puis individuel
15h35	15 min	War game en binôme	108	Binômes
15h50	10 min	Ce qu'on n'a pas vu et où le trouver, lundi matin	105, 109	Tous ensemble
16h00		Fin, ferme		

Ce qui est sacrifié, et je l'assume à voix haute à 9h00 comme à 15h50 : écrire l'instrumentation (TP 2), les recording rules et leurs tests (TP 3, les règles sont déjà en place grâce au rattrapage), les droits (2.30 à 2.33), le routage Alertmanager fin (TP 6 et 7), l'alerting Grafana en code (TP 8 partie 4), la sauvegarde (TP 9), Thanos (TP 10). Tout est dans le dépôt et dans les guides : faisable chez soi dans un Codespace.

Les points de contrôle. Si je suis en retard, je coupe dans cet ordre, sans remords :

1. Missions 7 et 8 (je les donne en correction directe, 3 minutes).
2. L'étape 2 du TP A (CPU par mode, mémoire) devient un bonus.
3. L'étape 3 du TP B (annotations) se fait en démonstration.
4. La partie 2 du TP C se fait en démonstration, les stagiaires regardent l'Inbox.

Je ne coupe **jamais** le war game : c'est le moment dont ils se souviendront.

Ce soir (mercredi) — 20 minutes

Pas à pas — pousser le rattrapage sur GitHub

Sur mon Mac, Terminal :

```
cd "/Users/yparent/Documents/PERSO/YSYCloud/FORMATION/PROMETHEUS GRAFANA/2026/formation-observabilite-lab"
git fetch ../formation-2026-jeudi.bundle formation-2026-formateur:formation-2026-formateur formation-2026:formation-2026 --update-head-ok --force
git push --force origin formation-2026 formation-2026-formateur
```

Puis sur github.com, branche `formation-2026` : le dossier `rattrapage/` doit apparaître, avec `appliquer.sh`, et `docs/stagiaire/Guide-stagiaire-Jour-3-express.pdf`.

Pas à pas — tester comme un stagiaire 10 min, à faire absolument

1. github.com → le dépôt → bouton vert **Code** → onglet **Codespaces** : je rouvre mon Codespace « stagiaire » (celui sur la branche `formation-2026`). S'il n'existe plus : **Create codespace on formation-2026**.

2. Dans le terminal du Codespace, les trois commandes :

```
git fetch origin
git checkout origin/formation-2026 -- rattrapage/
bash rattrapage/appliquer.sh
```

3. J'attends `10 / 10` cibles UP et Rattrapage terminé. Compter 2 à 4 minutes la première fois (construction de l'image de l'application).

4. Onglet **PORTS** → 3000 → Grafana s'ouvre, `admin / formation`. Explore → `up` → 10 lignes.

Si c'est bon ce soir, ce sera bon demain : le script est rejouable, il sauvegarde ce que les stagiaires ont déjà fait dans `rattrapage/sauvegarde-<date>/` et ne touche qu'aux fichiers du lab.

Pas à pas — préparer mon Codespace de démonstration

Mon Codespace de démo est sur la branche `formation-2026-formateur` (avec les corrigés).

```
git pull --ff-only || git reset --hard origin/formation-2026-formateur
bash rattrapage/appliquer.sh
cp solutions/jour-2/dashboards/*.json grafana/dashboards/
```

La dernière ligne fait apparaître, dans le dossier *Formation* de Grafana, les deux dashboards corrigés « TP 4 - Serveur Linux » et « TP 5 - Boutique en ligne ». Ce sont mes « résultats attendus » en direct, plus parlants que la capture de la slide. Je les ouvre une fois ce soir pour vérifier qu'ils se remplissent.

Pas à pas — le message à poster demain à 9h00

Je prépare ce message dans le chat de la session (Teams ou autre), prêt à envoyer :

Bonjour à tous ! Pour démarrer, dans le terminal de votre Codespace :

```
git fetch origin
git checkout origin/formation-2026 -- rattrapage/
bash rattrapage/appliquer.sh
```

Attendez "10 / 10 cibles UP" (2 à 4 minutes). Le guide du jour : `docs/stagiaire/Guide-stagiaire-Jour-3-express.pdf` (dans le dépôt, branche `formation-2026`)

8h30 — Vérification

- Mon Codespace de démo : `./lab.sh status`, Grafana ouvert, les deux dashboards corrigés se remplissent.
- Le deck ouvert en mode Présentateur sur la slide 41.
- Onglets du navigateur, dans l'ordre : le deck, Grafana (démo), Prometheus (démo), Inbox (démo), le dépôt GitHub.
- Le tableau blanc : j'y écris les trois commandes du rattrapage, en gros.

9h00 — Accueil et rattrapage 20 min

Ce que je dis

« Bonjour à tous. Hier, on a vu ensemble le concept d'instrumentation : une application qui expose elle-même ses métriques, avec une bibliothèque cliente. On s'est arrêté au moment où il fallait écrire le code. Je vous propose un changement de programme : vous êtes des gens d'infrastructure, pas des développeurs. Lundi, vous n'écrierez pas l'instrumentation de vos applications, ce sont vos développeurs qui le feront. Vous, vous allez **exploiter** ce qu'elles exposent : interroger, construire des tableaux de bord, alerter. Donc aujourd'hui, je vous donne l'application déjà instrumentée, et on passe la journée à piloter. Vous ne codez pas, vous pilotez. »

« Le programme : ce matin, on apprend à poser des questions à Prometheus, directement dans Grafana, puis on construit un vrai tableau de bord pour un serveur Linux. Cet après-midi, le tableau de bord de la boutique, avec le chiffre d'affaires et la santé du service, puis des alertes qui arrivent dans une boîte de réception. Et on finit par un jeu : vous allez casser le lab de votre voisin, et il devra trouver la panne avec vos tableaux de bord. »

J'envoie le message préparé dans le chat et je montre les trois commandes au tableau.

Pas à pas — le rattrapage, avec eux

Je le fais **en même temps qu'eux**, dans mon Codespace stagiaire projeté, pour qu'ils voient ce qui doit s'afficher.

1. Terminal du Codespace (s'il n'est pas visible : menu $\equiv$ → *Terminal* → *New Terminal*).
2. Les trois commandes. Je commente pendant que ça tourne : - « L'étape 1 sauvegarde ce que vous avez fait hier : rien n'est perdu. » - « L'étape 3 active les briques 1 à 6 dans `docker-compose.yml` : Prometheus, Grafana, l'application, le Node Exporter, les exporters, et l'Alertmanager qu'on utilisera cet après-midi. »
 - « L'étape 4 dépose le code de l'application terminé et la configuration de Prometheus complète. »
3. À la fin : `10 / 10 cibles UP`. Je fais lever la main de ceux qui l'ont.

Ceux qui n'ont pas 10/10 :

Symptôme	Cause probable	Remède
<code>error: pathspec 'origin/formation-2026'</code>	le fetch n'a pas été fait, ou leur dépôt n'est pas celui de la formation	<code>git remote -v</code> ; si besoin <code>git fetch origin formation-2026</code> puis refaire le checkout
Docker ne répond pas	Codespace qui vient de redémarrer	attendre 30 s, relancer le script
<code>toomanyrequests</code> pendant le démarrage	limite Docker Hub (image Redis ou Grafana)	<code>docker login</code> avec un compte Docker Hub gratuit, relancer
8 ou 9 cibles UP sur 10	un conteneur encore en démarrage	relancer le script : il est rejouable
<code>port is already allocated</code>	un binaire de mardi tourne encore, lancé d'un autre terminal	le script les arrête ; sinon fermer les anciens terminaux et relancer
Codespace inaccessible, supprimé		
en recréer un sur <code>formation-2026</code> (bouton Code), puis seulement <code>bash rattrapage/appliquer.sh</code>		

Pendant que les derniers terminent, je montre ce qu'ils auraient écrit.

Ce que je montre : le code qu'on n'écrit pas (slides 41 et 42)

J'ouvre `apps/shop-api/app.py` dans le Codespace projeté, et je fais défiler en montrant les cinq blocs qui étaient des TODO. Je ne lis pas le code : je montre qu'il est **court**.

« Voilà les cinq trous que vous deviez combler. Regardez : chaque métrique, c'est une déclaration de quatre lignes, et une ligne dans le code métier pour l'incrémenter. Ici `ORDERS.labels(payment_method=...).inc()` : à chaque commande validée, le compteur prend +1. C'est tout. Votre travail, lundi, ce sera de demander à vos développeurs : "est-ce que ton application expose `/metrics` ? Et est-ce qu'il y a au moins le débit, les erreurs et la durée ?" Si la réponse est oui, tout ce qu'on va faire aujourd'hui s'applique. »

Comparaison que je donne : **les compteurs du tableau de bord d'une voiture**. Le développeur installe les capteurs (l'instrumentation). Vous, vous concevez le tableau de bord et vous réglez les voyants (Grafana, les alertes). Vous n'avez pas besoin de savoir souder un capteur pour savoir qu'il faut un voyant d'huile.

9h20 — Mission 0 : la chasse aux métriques 15 min

Objectif. Revoir les quatre types de métriques sur du concret, sans slide.

Je projette la slide 22 (« Les quatre types ») trente secondes pour rappel, puis :

« Ouvrez le port 5001, et ajoutez `/metrics` à l'adresse. C'est exactement ce que Prometheus vient lire toutes les 15 secondes : du texte. Chaque ligne, c'est une série. Mission 0 dans votre guide : trouvez-moi un exemple de chacun des quatre types. Vous avez dix minutes. »

Astuce à donner : `Ctrl+F` dans la page, et chercher `# TYPE` : chaque métrique est précédée de sa ligne `# TYPE`.

Correction 5 min

Type	Exemple	Labels	Ce qu'elle mesure
Counter	<code>http_requests_total</code>	<code>method</code> , <code>route</code> , <code>status</code>	le nombre de requêtes depuis le démarrage
Counter	<code>shop_orders_total</code> , <code>shop_revenue_euros_total</code>	<code>payment_method</code>	commandes, chiffre d'affaires cumulés
Gauge	<code>shop_stock_units</code> , <code>shop_cart_items</code> , <code>http_requests_in_progress</code>	<code>product</code>	un niveau qui monte et descend
Histogram	<code>http_request_duration_seconds</code> (<code>_bucket</code> , <code>_sum</code> , <code>_count</code>)	<code>route</code> , <code>le</code>	la répartition des durées
Summary	<code>shop_payment_duration_seconds</code>	<code>quantile</code>	la durée du paiement, quantiles calculés dans l'application

Les questions :

1. **12 lignes** `_bucket` par route : 11 bornes (`0.005` à `10`) plus `+Inf`. `le="0.25"` veut dire « nombre de requêtes qui ont duré **moins de** 0,25 s » (*less or equal*). Les buckets sont **cumulatifs** : chaque marche contient les précédentes. Je dessine l'**escalier** : une marche par borne, chaque marche plus haute que la précédente, la dernière (`+Inf`) égale au total.
2. `_total` est la convention pour un **compteur** : il ne fait que monter. Une gauge n'a pas de suffixe.
3. `shop_app_info` vaut toujours 1 : l'information est **dans les labels** (`version`, `instance_name`). C'est le motif « info ». Utile pour afficher la version déployée dans un dashboard, et surtout pour savoir, après un incident, quelle version tournait.

Anecdote — la version qui tournait. Chez un client, un incident de nuit ; au post-mortem, personne n'est capable de dire quelle version tournait à 2 h du matin : trois déploiements dans la journée, des logs déjà purgés. Depuis, la première chose que je demande à une équipe, c'est une métrique `_info` avec la version. Une ligne de code, et une question de moins à chaque post-mortem.

9h35 — Missions 1 à 8 : PromQL dans Grafana Explore 55 min

Objectif. Les huit formes de PromQL qui couvrent l'essentiel des dashboards, pratiquées dans l'outil qu'ils utiliseront (Grafana), pas dans l'interface de Prometheus.

Pourquoi Explore et pas Prometheus. Parce que lundi, ils ouvriront Grafana, pas Prometheus. Explore, c'est le « brouillon » de Grafana : on tape une requête, on voit le résultat, sans créer de dashboard. Et ce qu'on met au point dans Explore se copie tel quel dans un panel cet après-midi.

Pas à pas — ouvrir Explore (je le fais projeté)

1. Grafana (port 3000) → menu de gauche → **Explore**.
2. En haut à gauche, la source : **Prometheus**.
3. Dans l'éditeur de requête, à droite, bascule **Builder / Code** : je choisis **Code**. Le Builder est pratique pour découvrir les noms de métriques, mais on apprend mieux en tapant.
4. Je tape `up`, **Shift+Entrée** (ou le bouton *Run query*). En dessous : un graphique et une table. Le sélecteur *Query type* (en bas de l'éditeur) : *Range* donne la courbe, *Instant* donne la valeur actuelle.
5. En haut à droite, la plage de temps : *Last 15 minutes*.

Le déroulé

Je fais **trois mini-exposés de 5 minutes** pendant les missions, au moment où la majorité en a besoin. Le reste du temps, je circule. Je corrige chaque mission au tableau quand les deux tiers l'ont faite, en tapant la réponse dans mon Explore projeté.

L'aide-mémoire des huit formes est dans leur guide, avant les missions. Je le montre et je dis :

« Vous n'avez pas à retenir PromQL par cœur. Ces huit formes, c'est 90 % de ce que vous écrirez dans votre vie. Pour chaque mission, trouvez la forme qui correspond et adaptez-la. C'est comme une recette de cuisine : on ne réinvente pas la pâte brisée, on la suit. »

Mission 1 — Qui est vivant ?

Correction.

```
up
up == 0
count by (job) (up)
```

- 10 séries. Chaque cible scrapée par Prometheus a une série `up` : 1 si le dernier scrape a réussi, 0 sinon. C'est la seule métrique que Prometheus crée lui-même.
- `up == 0` ne renvoie **rien** quand tout va bien : « No data ». Je le souligne : « Une comparaison en PromQL, ce n'est pas vrai ou faux, c'est un **tamis** : elle ne garde que ce qui passe. Rien ne passe, rien ne s'affiche. Et retenez ça pour cet après-midi : une alerte, c'est exactement cette requête-là. Si elle renvoie quelque chose, l'alerte sonne. »
- `count by (job)` : prometheus 1, node 1, shop-api 2, redis 1, pushgateway 1, blackbox-http 4.

Mini-exposé 1 — rate, le compteur kilométrique (slide 53, 5 min)

Au moment de la mission 2, quand ils découvrent la courbe qui ne fait que monter.

« Un compteur, c'est le compteur kilométrique de votre voiture : 124 532 km. Est-ce que ça vous dit si vous roulez vite ? Non. Ce qui vous intéresse, c'est le compteur de vitesse : combien de kilomètres **par heure**, maintenant. `rate`, c'est ça : il prend le compteur kilométrique sur les 5 dernières minutes, regarde de combien il a avancé, et divise par le temps. Résultat : une vitesse, en requêtes par seconde. »

Au tableau : une courbe en escalier qui monte (le compteur), et en dessous une courbe plate autour de 6 (le `rate`). Puis un redémarrage : le compteur retombe à zéro ; je montre que `rate` ne tombe pas, il détecte la remise à zéro et la corrige. « C'est pour ça qu'on ne calcule jamais une différence à la main sur un compteur. »

Et la règle d'or : **rate d'abord, sum ensuite**. « On calcule la vitesse de chaque voiture, puis on additionne les vitesses. L'inverse n'a pas de sens : additionner des compteurs kilométriques de voitures qui ont démarré à des moments différents. »

Mission 2 — Combien de requêtes ?

Correction.

```
rate(http_requests_total[5m])
sum(rate(http_requests_total{job="shop-api"}[5m]))
sum by (route) (rate(http_requests_total{job="shop-api"}[5m]))
```

Le total tourne autour de **6 à 7 requêtes par seconde** : c'est le générateur de trafic (réglé à 6 req/s, plus les scrapes et les sondes).

Mini-exposé 2 — sum by, le tableau croisé dynamique (slide 50, 5 min)

« Tout le monde a déjà fait un tableau croisé dynamique dans Excel ? `sum by (route)`, c'est exactement ça : "mets la route en ligne, et additionne tout le reste". Les instances, les méthodes, les codes : fusionnés. Il reste une ligne par route. `sum` tout court, sans `by`, c'est la case "Total général" en bas à droite. »

Au tableau : un tableau de 4 lignes (route, instance, valeur) et je montre les lignes qui fusionnent quand la colonne instance disparaît.

Mission 3 — Combien d'erreurs ?

Correction.

```
sum(rate(http_requests_total{job="shop-api", status=~"5.."}[5m]))
sum(rate(http_requests_total{job="shop-api", status=~"5.."}[5m]))
/
sum(rate(http_requests_total{job="shop-api"}[5m]))
```

1. Sans chaos : **rien** ou presque. L'application est saine.
2. Le taux d'erreur, entre 0 et 1. `~"5.."` : une regex, « commence par 5, suivi de deux caractères ».
3. Avec `./lab.sh chaos errors on` : 40 % des appels API échouent, mais le taux affiché monte **progressivement** pendant 5 minutes et plafonne vers 30 à 35 %. Deux questions à leur poser : pourquoi progressivement ? (la fenêtre de 5 minutes : au début, elle contient encore surtout des minutes saines ; c'est une moyenne glissante). Pourquoi pas 40 % ? (`/health` et `/metrics` ne tombent pas en panne, ils diluent le ratio).

Je leur fais penser à `chaos errors off` : ça sert à la mission suivante.

Mini-exposé 3 — le p95 et l'escalier des buckets (slide 54, 5 min)

« Le temps de réponse moyen, c'est l'ennemi. Si 95 clients sont servis en 50 ms et que 5 attendent 10 secondes, la moyenne est à 550 ms : elle ne décrit personne. Ni les 95 contents, ni les 5 furieux. Le p95, c'est : "95 % des requêtes sont plus rapides que ça". C'est la promesse qu'on peut faire au client. »

Puis l'escalier dessiné pendant la mission 0 : « `histogram_quantile` monte l'escalier jusqu'à la marche qui contient 95 % des requêtes, et fait une interpolation dans la marche. » La forme à recopier : `histogram_quantile(0.95, sum by (le) (rate(..._bucket[5m])))`. « Trois étapes : `rate` pour avoir des vitesses, `sum by (le)` pour fusionner les instances en gardant les marches, `histogram_quantile` pour lire l'escalier. Le `le` doit **toujours** rester dans le `by`, sinon il n'y a plus d'escalier. »

Mission 4 — Est-ce que c'est lent ?

Correction.

```
histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket[5m])))
histogram_quantile(0.95, sum by (route, le) (rate(http_request_duration_seconds_bucket[5m])))
```

1. Autour de 150 à 250 ms.
2. `/api/checkout` est la plus lente, autour de 400 ms (le paiement). Pour les rapides : `sort_desc(...)`.
3. `chaos latency on` multiplie les temps de traitement par 10 : le p95 passe au-dessus de la seconde en une à deux minutes.

Mission 5 — Et le business ?

Correction.

```
sum(rate(shop_revenue_euros_total[5m])) * 3600
sum(increase(shop_orders_total[1h]))
bottomk(3, avg by (product) (shop_stock_units))
```

1. `rate` donne des euros **par seconde**, `× 3600` donne des euros par heure. Autre réponse juste : `sum(increase(shop_revenue_euros_total[1h]))`, « combien sur la dernière heure ». Les deux diffèrent : l'un est une vitesse instantanée extrapolée, l'autre un cumul réel.
2. `increase` = « de combien le compteur a avancé », c'est `rate` `×` la durée. Si le lab tourne depuis moins d'une heure, `increase[1h]` extrapole : un nombre à virgule, c'est normal.
3. Chaque instance a son propre stock : on moyenne par produit, ou on garde les deux instances. Discussion utile : « Est-ce que c'est bien, un stock par instance ? Non, c'est un bug de conception de l'application. Et c'est la métrique qui le révèle. »

Mission 6 — Le serveur

Correction.

```
100 * (1 - avg(rate(node_cpu_seconds_total{mode="idle"}[5m])))
100 * (1 - node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes)
```

1. Le CPU, c'est la question piège du Node Exporter : il n'expose pas un pourcentage mais des **secondes** passées dans chaque mode, par cœur. `rate` sur le mode `idle` donne la part du temps où le cœur ne fait rien (entre 0 et 1). `1 -` donne la part occupée, `avg` fait la moyenne des cœurs.
2. `MemAvailable` et pas `MemFree` : Linux utilise la mémoire libre comme cache disque, et la rend quand on en a besoin. `MemFree` est toujours bas et fait peur pour rien.
3. `chaos cpu 120` lance, dans chaque instance de l'application, autant de processus qui tournent à vide qu'il y a de cœurs : le CPU monte vers 100 % pendant 2 minutes, puis redescend tout seul.

Anecdote — le serveur à 98 % de mémoire. Des années de tickets « mémoire critique » sur un serveur de base de données, parce que la supervision regardait `MemFree`. Le serveur allait très bien : Linux faisait son travail de cache. Le jour où on a basculé sur `MemAvailable`, les alertes ont disparu, et personne n'a regretté les 400 tickets par an.

Mission 7 — Vu de l'extérieur

Correction.

```
probe_success
sort_desc(probe_duration_seconds)
redis_up
redis_connected_clients
```

Les quatre sondes réussissent (1). La plus lente : `https://prometheus.io`, la seule qui sort sur Internet. Le message : « Le Blackbox voit votre service **comme un client** : de l'extérieur. Si `up` vous dit que Prometheus arrive à lire les métriques, `probe_success` vous dit que la page répond. Ce ne sont pas les mêmes pannes. »

`redis_up` = 1. Je pose la question qui servira au war game : « Si Redis tombe, `up` du job redis vaudra combien ? » Réponse : **toujours 1**. `up` parle de l'exporter, pas de Redis. C'est `redis_up` qui passe à 0.

Mission 8 — Le batch de la nuit

Correction.

```
time() - backup_last_success_timestamp_seconds
```

Quelques dizaines de secondes après `./lab.sh batch`. Le motif est universel pour les tâches planifiées : on ne surveille pas « le batch tourne », on surveille « la dernière réussite date de quand ». L'alerte naturelle : `> 26 * 3600` pour un batch quotidien.

Si j'ai du retard à 10h15

Je corrige les missions 7 et 8 directement, sans les faire faire : 3 minutes au lieu de 15.

10h30 — Pause 30 min

Avant la pause : « Gardez vos requêtes quelque part. Cet après-midi, vous les réutiliserez. »

11h00 — Grafana en 15 minutes

Slides 59 à 65, à **vive allure** : ils ont déjà vu Grafana hier en passant, ce qui compte, c'est la démo.

- Slide 60 (Grafana sous le capot) : « Grafana ne stocke rien. C'est une vitrine : il va chercher les données dans Prometheus à chaque affichage. »
- Slide 63 (Choisir la visualisation) : je la laisse affichée 1 minute, « c'est votre antisèche pour la journée ».
- Slide 64 (Ce qui rend un dashboard lisible) : les trois réglages. « Un dashboard sans unité, c'est une réunion où quelqu'un dit "on est à 0,4" et où personne n'ose demander 0,4 quoi. »

Anecdote — le dashboard du directeur. On m'avait demandé « un dashboard pour le directeur ». J'en ai livré un de 40 panels, magnifique. Il a regardé, et il a demandé : « Donc, ça va ou pas ? » Depuis, la première rangée de chaque dashboard répond à cette question, en quatre ou cinq chiffres avec des couleurs, et tout le détail est en dessous.

Pas à pas — démo d'un panel de bout en bout 7 min, projeté

Je construis le panel « CPU utilisé » du TP A devant eux, en expliquant chaque clic, puis je **jette** le dashboard (ils le refont eux-mêmes).

- Dashboards** → **New** → **New dashboard** → **Add visualization** → source *Prometheus*.
- Éditeur de requête en bas, mode **Code** : `100 * (1 - avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[$__rate_interval])))`. « `$__rate_interval` à la place de `[5m]` : Grafana choisit la fenêtre selon le zoom. Si vous regardez 7 jours, il ne va pas calculer des moyennes sur 5 minutes. »
- À droite, en haut, le type : je commence en *Time series* (le défaut), puis je passe à **Gauge**. « Même requête, autre visualisation. »
- Options à droite, champ de recherche en haut : je tape `unit` → *Percent (0-100)*. Puis `min` 0, `max` 100.
- Thresholds** : vert par défaut, j'ajoute 70 orange et 90 rouge.
- Titre : `CPU utilisé`. En haut à droite : **Back to dashboard**.
- Dans le terminal : `./lab.sh chaos cpu 60`. Je règle le rafraîchissement automatique (menu à côté du bouton *Refresh*, 10 s). La jauge passe à l'orange puis au rouge : effet garanti.

« Voilà tout le métier : une requête, une visualisation, une unité, des seuils. Vous allez le faire dix fois ce matin. »

11h15 — TP A : le dashboard d'un serveur Linux 65 min

Objectif. Un dashboard paramétrable (une variable), lisible, avec cinq types de visualisation différents : Stat, Gauge, Time series, Bar gauge, State timeline.

Le choix pédagogique. Les requêtes sont **données** dans leur guide. Le travail est dans Grafana : visualisation, unité, seuils, légendes, overrides. Ceux qui veulent chercher la requête eux-mêmes peuvent cacher la colonne.

Slides 66 et 67 : la mise en situation, puis le résultat attendu. Mieux : j'ouvre le dashboard corrigé « TP 4 - Serveur Linux » dans mon Grafana de démo et je le laisse projeté pendant tout le TP. « Le vôtre aura quelques panels de moins, c'est normal. »

Pas à pas — la variable (je la montre, 3 min)

C'est la seule étape que je montre, parce que c'est la plus déroutante :

- Dans le dashboard vide, mode édition : barre latérale **Add** → **Variable** (ou **Settings** → **Variables** → **New variable**).
- Variable type** : Query. **Name** : `instance`. **Label** : `Serveur`.
- Query** : `label_values(node_uname_info, instance)`.
- Cocher **Multi-value** et **Include All option** ; **Custom all value** : `.*`.
- En bas, **Preview of values** : `node-exporter:9100`. **Back to dashboard**.

6. La liste déroulante « Serveur » apparaît en haut du dashboard.

« Dans chaque requête, on ajoutera `instance=~"$instance"`. Grafana remplace `$instance` par ce qui est choisi dans la liste. Le `≈` au lieu de `=`, parce que quand on choisit "All", ça devient `.*`, une regex. Ici, on n'a qu'un serveur ; chez vous, ce sera 200, et le même dashboard servira pour les 200. »

Pendant le TP : les pièges, dans l'ordre où ils arrivent

Piège	Symptôme	Ce que je dis
Requête en <i>Range</i> sur un Stat	ça marche, mais c'est lent	« Pour une valeur actuelle : <i>Instant</i> (sous la requête, <i>Options</i> → <i>Type</i>). »
Unité oubliée	0.137 au lieu de 13,7 %	« Percent (0-100) si la requête fait <code>× 100</code> , Percent (0.0-1.0) sinon. »
Uptime en secondes brutes	10234	Unit <i>duration</i> (<i>dt durations</i>) : « 2 hours, 50 minutes ».
Panel 4, trois requêtes	une seule courbe	« Le bouton + Query sous la première requête. »
Panel 6, sans <code>scalar()</code>	No data	voir ci-dessous
Bar gauge disque vide	tous les montages exclus	vérifier l'antislash : dans Grafana, on tape <code>tmpfs overlay squashfs</code> sans <code>\</code>
State timeline tout vert, pas de texte	pas de mappings	<i>Value mappings</i> → <i>Add value mapping</i> : 1 → UP vert, 0 → DOWN rouge
Panel 10 « No data »	pas de <code>or vector(0)</code>	voir ci-dessous
Ils oublient de sauvegarder	tout est perdu au rechargement	« Save dashboard , toutes les 10 minutes. » (Ctrl+S / Cmd+S)

La question du panel 6 (`scalar`). À gauche de la division, une série par mode, avec un label `mode`. À droite, `count(...)`, une seule série sans aucun label. PromQL apparie les séries par labels identiques : aucune ne correspond, résultat vide. `scalar()` transforme la série de droite en simple nombre, qui divise tout le monde. Comparaison : « C'est comme vouloir faire une RECHERCHE dans Excel avec une colonne qui n'existe pas dans l'autre tableau. »

La question du panel 10 (`or vector(0)`). `count(up == 0)` quand tout va bien : le tamis ne laisse rien passer, `count` de rien, c'est rien, donc « No data ». `or vector(0)` : « s'il n'y a rien, prends 0 ». Un 0 vert est rassurant, un « No data » est inquiétant : on ne sait pas si c'est « aucune panne » ou « la requête est cassée ».

Le test du TP (à faire faire vers 12h05) : `docker stop shop-api-2`. En 15 à 30 secondes : la State timeline passe au rouge pour `shop-api / shop-api-2:5000`, et le panel « Cibles en panne » passe à 1 et au rouge. Question à poser : « La sonde Blackbox de `shop-api-2`, elle, est-elle rouge dans la State timeline ? » Non : `up` de la sonde reste à 1, parce que c'est le **Blackbox** que Prometheus scrape, et lui va bien. C'est `probe_success` qui passe à 0 (mission 7). `docker start shop-api-2`.

Point de contrôle à 12h00

Si la majorité n'a pas fini l'étape 1 : on saute l'étape 2 (elle devient un bonus) et on fait l'étape 3, qui est la plus spectaculaire (le test `docker stop`).

Pas à pas — le joker, pour celui qui est complètement perdu

Uniquement pour un stagiaire en difficulté, pour qu'il puisse suivre la suite :

```
git fetch origin
git checkout origin/formation-2026-formateur -- solutions/jour-2/dashboards/
cp solutions/jour-2/dashboards/*.json grafana/dashboards/
```

Dix secondes plus tard, les deux dashboards corrigés sont dans le dossier *Formation* de son Grafana. Il peut les ouvrir, les étudier, les modifier.

12h20 — Le dashboard de la communauté, puis débrief 10 min

Pas à pas — importer Node Exporter Full

1. **Dashboards** → **New** → **Import**.
2. *Find and import dashboards...* : `1860` → **Load**.
3. En bas, la source *Prometheus* → **Import**.

(Si l'import par ID échoue, c'est qu'un proxy bloque `grafana.com` : ce n'est pas le cas dans Codespaces. Sur un poste d'entreprise, on télécharge le JSON sur `grafana.com/grafana/dashboards` et on utilise *Upload dashboard JSON file*.)

« Voilà le dashboard le plus téléchargé de `grafana.com`. Des dizaines de panels. Il est très bien pour enquêter. Maintenant, la question : si vous êtes d'astreinte, à 3 h du matin, et qu'on vous appelle, vous ouvrez lequel ? Le vôtre, avec dix panels et des couleurs, ou celui-là ? »

Le message : **on commence toujours par un dashboard communautaire** (gratuit, éprouvé), et on construit **son propre dashboard de synthèse** pour la question « est-ce que ça va ? ». Rien n'empêche de les relier : c'est l'étape 3 du TP B (les liens).

Débrief rapide : un tour de table, « le réglage que vous ne connaissiez pas ce matin et que vous utiliserez ». Réponses fréquentes : les unités, les value mappings, la variable.

12h30 — Déjeuner

« Laissez vos Codespaces ouverts. S'ils se mettent en veille, ils redémarrent avec la stack au retour : il faut juste attendre 30 secondes. »

13h30 — TP B : le dashboard de la boutique 60 min

Objectif. Un dashboard qui parle à deux publics (métier et technique), les trois signaux RED, la heatmap, et les annotations qui montrent les incidents sur les courbes.

Slides 68 et 69, puis le dashboard corrigé « TP 5 - Boutique en ligne » projeté dans mon Grafana de démo.

Ce que je dis en lançant le TP

« Ce matin, vous avez fait un dashboard d'infrastructure : CPU, mémoire, disque. Vos utilisateurs s'en moquent. Ce qui les intéresse, c'est : est-ce que je peux commander, est-ce que c'est rapide. Et ce qui intéresse votre direction, c'est : combien on vend. Ce dashboard met les deux sur le même écran. »

RED au tableau. J'écris verticalement R, E, D :

- **Rate** : combien de demandes (le débit, `rate(http_requests_total)`) ;
- **Errors** : combien échouent (le ratio de la mission 3) ;
- **Duration** : combien de temps (le p95 de la mission 4).

« Pour n'importe quel service, que ce soit une API, une base, une file de messages, ce sont les trois premières questions. Si les trois sont bonnes, les utilisateurs sont contents. C'est la méthode formalisée par Tom Wilkie (aujourd'hui chez Grafana Labs), et c'est la meilleure grille de départ que je connaisse. »

Comparaison : **le restaurant**. Rate : combien de clients entrent. Errors : combien de plats reviennent en cuisine. Duration : combien de temps entre la commande et l'assiette. Le patron regarde ces trois chiffres ; la température du four (le CPU), c'est pour le cuisinier.

Pendant le TP : les pièges

Piège	Ce que je dis
Stat « Chiffre d'affaires » qui affiche un nombre énorme	« Vous avez mis le compteur brut : c'est tout ce qui a été vendu depuis le démarrage. On veut une vitesse, <code>rate</code> × 3600. » (c'est la question du guide)
Pie chart avec une seule part	la légende n'a pas <code>{{payment_method}}</code> , ou la requête n'a pas le <code>by</code>
Stock : 12 barres	<code>pas d' avg by (product)</code> , les deux instances sont affichées
Heatmap toute grise	<i>Format : Heatmap</i> oublié sous la requête (<i>Options</i> → <i>Format</i>) ; puis <i>Calculate from data : No</i> dans les options
Taux d'erreur à 0 en permanence	normal sans chaos ; c'est le moment de lancer <code>chaos errors on</code>
Seuils du taux d'erreur invisibles	<i>Thresholds</i> à 0.05 et <i>Show thresholds : As lines and filled regions</i>

La heatmap, je la montre pendant le chaos latency : « Chaque colonne, c'est une minute. Chaque case, c'est une marche de l'escalier. La couleur, c'est le nombre de requêtes dans la marche. Quand on active la latence, vous voyez toute la masse **monter** : c'est la répartition qui se déplace, pas juste une moyenne. »

Les annotations (étape 3) : l'effet waouh de l'après-midi. Quand le chaos errors s'active, un trait rouge vertical apparaît sur **toutes** les courbes au même instant.

« En post-mortem, la première question est toujours : "qu'est-ce qui a changé à ce moment-là ?" Si vos déploiements sont annotés sur vos dashboards, vous avez la réponse sans chercher. C'est la fonction la plus sous-utilisée de Grafana. »

La requête d'annotation, `changes(shop_chaos_mode[1m]) > 0` , s'explique en une phrase : « dès qu'une gauge change de valeur, on met un trait ».

Pour les rapides : dashboards as code (démon en 3 minutes si j'ai le temps)

Export → Export as code → copier le JSON → le coller dans `grafana/dashboards/tp-b-boutique.json` dans l'éditeur du Codespace → 10 secondes plus tard, il apparaît avec un cadenas dans la liste.

« Votre dashboard est maintenant un fichier. On peut le mettre dans Git, le relire en merge request, le déployer sur dix Grafana. C'est comme ça qu'on travaille en production : personne ne modifie les dashboards à la main. »

Point de contrôle à 14h20

Si la majorité n'a pas fini l'étape 2 : je fais l'étape 3 (annotations) en démonstration projetée, et ils la recopient s'ils ont le temps après la pause.

14h30 — Pause 30 min

Avant la pause : « Après la pause, on fait sonner des alarmes. Remettez votre lab en ordre : `./lab.sh chaos reset`. »

15h00 — TP C : alerter 35 min

Les concepts en 5 minutes (slides 78, 81, 94)

- **Slide 78, Mauvaise alerte, bonne alerte.** « Une bonne alerte réveille quelqu'un pour quelque chose que **les utilisateurs sentent**, et sur laquelle il peut **agir**. Le CPU à 90 %, ce n'est pas une alerte : c'est une information. Le taux d'erreur à 10 %, c'est une alerte. »

Anecdote — les 400 notifications par jour. (slide 79) Une équipe recevait 400 notifications par jour. Plus personne ne les lisait ; le jour où la vraie panne est arrivée, elle était la 237e de la journée. Après nettoyage : 12 alertes, toutes sur des symptômes. L'astreinte est redevenue vivable, et les pannes sont détectées plus vite.

- **Slide 81, Le cycle de vie.** Au tableau, trois cases : **Inactive** → **Pending** → **Firing**. « La condition devient vraie : Pending. Elle reste vraie pendant la durée `for` : Firing, et la notification part. Le `for`, c'est l'anti-faux positif : un pic d'une seconde ne réveille personne. »
- **Slide 94, Quand utiliser quoi.** Prometheus + Alertmanager pour les alertes sur les métriques, versionnées dans Git ; Grafana pour les alertes sur d'autres sources (SQL, logs, cloud) ou pour les équipes qui ne toucheront jamais un YAML. « Jamais la même alerte des deux côtés, sinon vous recevez tout en double. »

Partie 1 — Une alerte Prometheus de bout en bout 10 min

La règle `TargetDown` existe déjà dans `prometheus/rules/alerts.yml` ; le rattrapage a branché Prometheus sur l'Alertmanager. Je le fais **en même temps qu'eux**, projeté.

Pas à pas — suivre TargetDown

1. Trois onglets côte à côte : Prometheus (9090) → **Alerts** ; Alertmanager (9093) ; Inbox (8080).
2. Je montre la règle dans Prometheus → **Alerts** : `TargetDown`, `up == 0`, `for: 1m`, `Inactive`, en vert.
3. Terminal : `docker stop shop-api-1`. Je note l'heure au tableau.
4. 15 à 30 s : `Pending` (orange) dans Prometheus.
5. 1 min plus tard : `Firing` (rouge). L'alerte apparaît dans l'Alertmanager.
6. 30 s plus tard (le `group_wait`) : la notification arrive dans l'Inbox.
7. `docker start shop-api-1`. L'alerte disparaît de Prometheus en 30 s ; la notification `resolved` arrive dans l'Inbox jusqu'à 5 minutes plus tard (le `group_interval`).

La question du guide : pourquoi 1 min 30 à 2 min de délai ? Scrape (jusqu'à 15 s) + évaluation (jusqu'à 15 s) + `for` (1 min) + `group_wait` (30 s). Ce n'est pas un défaut, c'est **voulu** : on échange un peu de réactivité contre beaucoup moins de fausses alertes. Comparaison : **le détecteur de fumée** qui sonnerait au premier grille-pain. Tout le monde finirait par enlever la pile.

Partie 2 — Une alerte Grafana sur le taux d'erreur 20 min

Ils la font seuls avec le guide. Je montre juste le contact point, parce que l'URL est déroutante.

Pas à pas — le contact point (projeté, 2 min)

1. **Alerting** → **Notification configuration** → **Contact points** → **+ Create contact point**.
2. **Name** : `inbox-grafana`. **Integration** : **Webhook**. **URL** : `http://inbox:8080/webhook/grafana`. Je dis : « Pourquoi `inbox` et pas `localhost` ? Parce que c'est Grafana qui envoie, depuis son conteneur. Pour lui, `localhost`, c'est lui-même. Les conteneurs se parlent par leur nom, sur le réseau `formation-monitoring`. »
3. **Test** → **Send test notification** → dans l'Inbox, une notification « TestAlert » arrive.

4. Save contact point.

Chez eux, dans la vraie vie : l'intégration **Microsoft Teams** à la place du Webhook, avec l'URL d'un workflow Teams. C'est le TP 7 du guide du jour 3, à faire chez soi.

Pendant la partie 2 : les pièges

Piège	Ce que je dis
Preview en erreur de format, ou des courbes au lieu d'une valeur	la requête est en <i>Range</i> : passer en Instant
La condition ne se déclenche pas	le seuil : la requête est en pourcentage (× 100), donc <code>5</code> , pas <code>0.05</code>
Une seule alerte au lieu d'une par instance	le <code>sum by (instance)</code> a été oublié
Pas de notification	le contact point n'est pas choisi dans la règle, ou le chaos n'est pas actif
Tout est long	<i>Pending period</i> 1 min + intervalle 1 min : il faut 2 à 3 minutes, patience

Le résultat : dans l'Inbox, deux notifications « Boutique - taux d'erreur élevé », une par instance ; sur le dashboard TP B, l'état de l'alerte apparaît sur le panel « Taux d'erreur ». `./lab.sh chaos errors off`.

La question du guide (Prometheus ou Grafana ?) : la réponse attendue est « ça dépend », avec les critères de la slide 94. Chez Apicil, avec des équipes infra : Prometheus pour tout ce qui est métrique et doit être versionné, Grafana si l'alerte porte sur une base SQL ou des logs.

15h35 — War game en binôme 15 min

Slide 108. C'est le moment le plus joyeux de la journée : je le présente comme un jeu.

« Par deux, sur un seul écran. L'un se retourne. L'autre tire une carte, tape la commande, ferme le terminal. Celui qui s'est retourné a cinq minutes pour trouver la panne, avec vos dashboards et Explore, rien d'autre. Pas le terminal, pas les logs. Puis on remet en ordre, et on inverse. Remplissez la grille : je veux savoir **quoi, où, depuis quand**. »

Règle à ajouter : **avant de jouer, décochez l'annotation « Chaos »** en haut du dashboard TP B, sinon le trait rouge avec « Chaos errors » donne la réponse.

Je chronomètre au tableau : 5 minutes par manche, 2 manches, plus la remise en ordre.

Les solutions des cartes

Carte	Commande	Où ça se voit	Le piège
1	<code>./lab.sh chaos latency on</code>	TP B : p95 / p99, heatmap qui monte	aucun, c'est la carte facile
2	<code>./lab.sh chaos errors on</code>	TP B : taux d'erreur à 30 % ; alerte Grafana dans l'Inbox	la fenêtre de 5 min : ça monte lentement
3	<code>curl ... 5002/chaos/errors/on</code>	taux d'erreur à 15-20 % seulement	il faut filtrer la variable <code>instance</code> sur <code>shop-api-2</code> pour voir 40 %
4	<code>./lab.sh traffic 40</code>	débit × 6, CPU en hausse, chiffre d'affaires qui s'envole	ce n'est pas une panne : c'est le Black Friday. Bonne réponse : « rien de cassé, beaucoup de clients »
5	<code>./lab.sh chaos cpu 180</code>	TP A : jauge CPU rouge, CPU par mode <i>user</i> ; la latence monte un peu	la saturation n'est pas encore une panne ; le chaos s'arrête seul au bout de 3 min (<code>chaos reset</code> ne l'arrête pas)
6	<code>docker stop shop-api-1</code>	TP A : State timeline, « Cibles en panne » = 1 ; alerte TargetDown dans l'Inbox ; <code>probe_success</code> à 0	le débit total baisse peu : l'autre instance absorbe
7	<code>docker stop redis</code>	Explore : <code>redis_up</code> = 0	<code>up{job="redis"}</code> reste à 1 : l'exporter va bien, c'est Redis qui est mort
8	<code>./lab.sh chaos leak on</code>	Explore : <code>process_resident_memory_bytes{job="shop-api"}</code> qui grimpe	aucun dashboard ne le montre : c'est une fuite mémoire, elle finira en crash dans quelques heures

Le débrief (3 minutes) : « Qui a trouvé en moins de deux minutes ? Avec quel panel ? » Et la leçon des cartes 7 et 8 : « Un dashboard ne montre que ce qu'on a pensé à y mettre. Explore sert à tout le reste. »

15h50 — Clôture 10 min

Ce qu'on n'a pas vu, et où le trouver

Slide 105 (Thanos), une minute :

« Prometheus garde quinze jours par défaut, sur un seul serveur. Quand vous voudrez un an d'historique, ou une vue sur plusieurs Prometheus, c'est Thanos, ou Mimir chez Grafana Labs. Le TP complet est dans le guide du jour 3, il tourne dans un Codespace. »

Puis la dernière page de leur guide du jour : le tableau « Ce qu'on n'a pas eu le temps de faire ». « Tout est dans le dépôt. Votre Codespace, vous pouvez le recréer quand vous voulez, gratuitement, sur la branche `formation-2026`. Les guides des trois jours sont dans `docs/stagiaire/`. »

Lundi matin (slide 109)

« Lundi matin, ne faites pas tout. Choisissez **un** service. Un seul. Vérifiez qu'il expose des métriques, ou qu'il existe un exporter. Faites-le scraper. Construisez-lui une row RED : débit, erreurs, latence. Puis **une** alerte, sur un symptôme que vos utilisateurs sentiraient. C'est tout. Dans un mois, vous aurez dix services, parce que les collègues viendront vous demander le même. »

Je leur fais remplir la dernière ligne de leur guide (« Mon service : ... ») et je fais un tour de table express : un service chacun, à voix haute. Ça engage.

Le mot de la fin, à 15h58 :

« Mardi matin, vous ne saviez pas ce qu'était une série temporelle. Aujourd'hui, vous avez interrogé, visualisé, cassé, diagnostiqué et alerté sur une application que vous n'aviez jamais vue. C'est exactement ce que vous ferez lundi. Merci à tous. »

16h00 : fin.